

Exercise 2.3. 12.7 is equal to 127/10, where all numbers are decimal. Can you imagine it as the value of a binary fraction? And the number 5.5?

Exercise 2.4. How would you write the following XML snippet in a Lua string?

```
<![CDATA [  
Hello world  
]]>
```

Use at least two different methods.

Exercise 2.5. Let's say you need to write a long after-a sequence of arbitrary bytes as a string constant in Lua. How do you do it? Pay attention to readability, maximum line length and performance.

Exercise 2.6. Consider the following code:

```
a = {}; aa = a
```

What will be the value of `aaaa` ? Any `a` in this sequence is somehow different from the rest?

Now add the following line to the previous code:

```
aaaa = 3
```

What will be the value of `aaaa` now ?

CHAPTER 3

Expressions

Expressions represent values. Expressions in Lua include numeric constants and string literals, variables, unary and binary operations and function calls. Expressions also include- contain non-standard function definitions and constructors for tables.

3.1. Arithmetic operators

Lua supports the standard arithmetic operators: binary-
nye '+' (addition), '-' (subtraction), '*' (multiplication), '/' (div-
number), '^' (exponentiation), '%' (remainder of division), and unary
'-' (change sign). All of them work with floating-point numbers.

point. For example, $x^{0.5}$ calculates the square root of x ,
and $x^{(-1/3)}$ computes the inverse of the cube root of x .

The following rule defines the modulus operator:

```
a% b == a - math.floor (a / b) * b
```

For integer operands, it has a standard value, and re-

The result has the same sign as the second operand. For real
operands, it has some additional features. On-

for example, $x\%$ 1 gives the fractional part of x , and $xx\%$ 1 gives the integer
part. Ana-

logical $xx\%$ 0.01 gives x with exactly two decimal places after
comma:

```
x = math.pi  
print (x - x% 0.01) -> 3.14
```

As another example of using the remainder operator

division, consider the following example: let's say you want to know
will the vehicle after turning at a given angle

move in the opposite direction. If the angle is given in degrees, then
you can use the following formula:

```
local tolerance = 10  
function isturnback (angle)  
  angle = angle% 360  
  return (math.abs (angle - 180) < tolerance)  
end
```

This definition works even for negative angles:

```
print (isturnback (-180)) -> true
```

If you want to work in radians instead of degrees, we just
let's change the constants in the functions:

```
local tolerance = 0.17  
function isturnback (angle)  
  angle = angle% (2 * math.pi)  
  return (math.abs (angle - math.pi) < tolerance)  
end
```

All we need is an $angle\%$ (2 * math.pi) operation to convert

any angle to the interval $[0, 2\pi)$.

3.2. Comparison Operators

Lua provides the following comparison operators:

`< <= > >= == ~=`

All of these operators are always boolean.

The `==` operator tests for equality; operator `~=` is negation equality. We can use both of these operators to any two values. If the values are of different types, then Lua assumes that they are not equal. Otherwise Lua compares them accordingly their type. The **nil** value is equal only to itself.

Lua compares tables and objects of type userdata by reference, i.e. two such values are considered equal only if they are the same object. For example, after doing the following code:

```
a = {}
ax = 1
ay = 0
b = {}
bx = 1
by = 0
c = a
```

we get `a == c` , but `a ~= b` .

We can only apply order operators to a pair of numbers, or a couple of lines. Lua compares strings alphabetically, following the convention

The locale set for Lua. For example, for the Portuguese locale Latin-1 we get `"acai" < "açaí" < "acorde"` . Values of types from-personal from strings and numbers, can only be compared for equality (and inequality).

When comparing values of different types, you need to be careful nym: remember that `"0"` is different from `0` . Moreover, `2 < 15` is obvious true, but `"2" < "15"` is false. In case you are trying to compare a string and a number, for example `2 < "15"` , an error occurs.

3.3. Logical operators

The logical operators are **and** , **or** and **not** . As well as managing constructs, logical operators treat **false** and **nil** as false and all others as true values. **And** return operator

gives its first operand if it is false, otherwise it returns its second operand. The **or** operator returns its first operand if it is not false; otherwise it returns its second operand:

```
print (4 and 5)
-> 5
print (nil and 13)
-> nil
print (false and 13)
-> false
print (4 or 5)
-> 4
print (false or 5)
-> 5
```

Both operators (**and** and **or**) use shorthand evaluation, then there they calculate their second operand only when it is necessary. This ensures that expressions like `(type (v) == "table" and v.tag == "h1")` will not cause errors in their computation: Lua will not try to compute `v.tag` when `v` is not a table.

A useful construct in Lua is `x = x or v` , which is equivalent to the following code:

```
if not x then x = v end
```

That is, the `x` value is set equal to the default `v` if `x` is undefined (assuming **x is not false**).

Another useful construct is `(a and b) or c` , or simply `a and b or c` , as the operator **and** has a higher preference than **or** . It is equivalent to the expression `a ? B: c` in C, when provided that `b` is not false. For example, we can choose the maximum from two numbers `x` and `y` using the following operator:

```
max = (x > y) and x or y
```

When `x > y` , then the first expression in the **and** operator is true, to this it returns its second value (`x`), which is always tiny (since it's a number), and then the **or** operator returns its first operand, `x` . If the expression `x > y` is false, then the result of the operator **and** is also false, and so the **or** operator returns its second operand, `y` .

The not operator always returns a boolean value:

```
print (not nil)
-> true
print (not false)
-> true
print (not 0)
-> false
```

```
print (not not 1)
-> true
print (not not nil)
-> false
```

3.4. Concatenation

Lua refers to the concatenation operator as `..` (dots). If operand is a number, Lua will convert it to a string. (Some languages use the `+` operator for concatenation, but Lua `3 + 5` differs from `3..5`.)

```
print ("Hello" .. "World")
-> Hello World
print (0 .. 1)
-> 01
print (000 .. 01)
-> 01
```

Remember that strings in Lua are immutable values.

The concatenation operator always creates a new string without changing its

their operands:

```
a = "Hello"
print (a .. "World")
-> Hello World
print (a)
-> Hello
```

3.5. Length operator

The length operator works with strings and tables. With strings it gives the number of bytes per line. With tables, it returns the length of the *pos-*

research presented in the table.

There are several common idioms associated with the length operator to work with sequences.

```
print (a[#a]) - prints the last element of the sequence 'a'
a[#a] = nil - removes the last element
a[#a + 1] = v - adds 'v' to the end of the list
```

As we saw in the previous chapter, the length operator is not let's say for lists with holes (`nil`). It only works for software

sequences, which we defined as lists without holes.

More precisely, a *sequence* is a table where the keys are give a sequence $1, \dots, n$ for some n . (Remember, that any **nil** key is not actually in the table.)

In particular, a table without numeric keys is a sequential length zero.

Over the years, there have been many proposals to expand the meaning of opera-

lengths to lists with holes, but this is easier said than done.

The problem is that since a list is a table, the concept "Length" is somewhat vague. For example, consider a list produced by the following code snippet:

```
a = {}  
a [1] = 1  
a [2] = nil - does nothing, since a [2] is already nil  
a [3] = 1  
a [4] = 1
```

It is easy to say that the length of this list is four and it has a hole in index 2. However, what about the following example?

```
a = {}  
a [1] = 1  
a [10000] = 1
```

Should we treat this a as a list with 10,000 elements tam where 9998 elements are **nil** ? Now let the program do following:

```
a [10000] = nil
```

What happened to the length of the list? Should it be 9999, since the program removed the last item? Or maybe 10,000, because the program just changed the value of the last element cop to **nil** ? Or should the length be 1?

Another common suggestion is to do so so that the # operator returns the number of elements in the table. This semantic

the teak is clear and well defined, but of no benefit.

Let's consider all the previous examples and imagine how much A similar operator would be useful for algorithms that work with lists or arrays.

Even more problematic are the **nil** values at the end of the list. Ka-

What should be the length of the following list?

```
a = {10, 20, 30, nil, nil}
```

Recall that for Lua, a **nil** field is no different from accompanying field. Thus, the previous table is indistinguishable from $\{10,20,30\}$; its length is 3, not 5.

You might think that **nil** at the end of the list is a special case.

However, many lists are built by adding elements, one after another. Any list with holes built like this thus, simply obtained by adding **nil** to its end.

Many of the lists we use in our programs are sequences (for example, a line in a file cannot be **nil**), and so most of the time the length operator is not passive for use. If you really have to work with lists with holes, then you'd better clearly remember the length somewhere list.

3.6. Operator Priorities

The operator precedence in Lua is given in the table below, from the oldest to the lowest:

^
not # - (unary)
* / %
+ -
..
<> <=> = ~ ==
and
or

All binary operators are left associative, except '^' (exponentiation) and '..' (concatenation), which are associative to the right. Therefore the following expressions on the left are equivalent

expressions on the right:

$a + i < b / 2 + 1$
<-->
 $(a + i) < ((b / 2) + 1)$
 $5 + x ^ 2 * 8$
<-->
 $5 + ((x ^ 2) * 8)$
 $a < y$ and $y <= z$ <-->
 $(a < y)$ and $(y <= z)$
 $-x ^ 2$

```
<-->
```

```
- (x ^ 2)
```

```
x ^ y ^ z
```

```
<-->
```

```
x ^ (y ^ z)
```

When in doubt, always use parentheses. It's easier than look in the manual, and most likely later when you read this code, you will have doubts again.

3.7. Table constructors

Constructors are expressions that create and initialize tables. They are the hallmark of Lua and one of its most useful and versatile mechanisms.

The simplest constructor is an empty constructor, `{}`, which creates an empty table; we've seen this before. Constructors also initialize lists. For example, the operator

```
days = {"Sunday", "Monday", "Tuesday", "Wednesday",  
"Thursday", "Friday", "Saturday"}
```

will initialize `days[1]` to "Sunday" (the first element constructor has index 1, not 0), `days[2]` is "Monday" etc.:

```
print(days[4]) -> Wednesday
```

Lua also offers special syntax for initialization tables by fields, as in the following example:

```
a = {x = 10, y = 20}
```

This line is equivalent to the following commands:

```
a = {}; ax = 10; ay = 20
```

The original expression is simpler and faster because Lua immediately creates

It has a table with the correct size.

Regardless of which constructor we used to create a table, we can always add and remove fields from her:

```
w = {x = 0, y = 0, label = "console"}
```

```
x = {math.sin(0), math.sin(1), math.sin(2)}
```

```
w[1] = "another field" - add key 1 to table 'w'
```

```
xf = w
```

```
- add key "f" to table 'x'
```

```
print(w["x"])
```

```
-> 0
```

```
print(w[1])
```

```
-> another field
```

```
print (xf [1])
-> another field
wx = nil
- remove field "x"
```

However, creating the table right away with the correct boot more efficiently and clearly.

We can mix these two initialization styles (list and by fields) in the same constructor:

```
polyline = {color = "blue",
thickness = 2,
npoints = 4,
{x = 0, y = 0},
- polyline [1]
{x = -10, y = 0},
- polyline [2]
{x = -10, y = 1},
- polyline [3]
{x = 0, y = 1}
- polyline [4]
}
```

The example above also shows how contributions can be nest constructors one into another to represent more complex data structures. Each of the polyline [i] elements is a table, representing a record:

```
print (polyline [2] .x)
-> -10
print (polyline [4] .y)
-> 1
```

These two forms of constructor have their limitations. For instance, you cannot initialize fields with negative indices or with indices that are not identifiers. For such goals have a different, more general format. In this format, we clearly write we use the index as an expression between square brackets:

```
opnames = {[ "+" ] = "add", [ "-" ] = "sub",
[ "*" ] = "Mul", [ "/" ] = "div"}
i = 20; s = "-"
a = {[i + 0] = s, [i + 1] = s..s, [i + 2] = s..s..s}
print (opnames [s])
-> sub
print (a [22])
-> ---
```

This syntax is more awkward, but also more general: considered previously constructor forms are special cases of this more general syntax. The constructor {x = 0, y = 0} is equivalent to

$\{["X"] = 0, ["y"] = 0\}$, and the constructor $\{"r", "g", "b"\}$ is equivalent to $\{[1] = "r", [2] = "g", [3] = "b"\}$.

You can always put a comma after the last entry in constructor. These commas are optional:

`a = {[1] = "red", [2] = "green", [3] = "blue"},`

This frees programs that generate Lua constructors from the need to process the last element in a special way.

Finally, you can always use dot c in the constructor comma instead of comma. I usually use semicolons to de-division of various sections in the constructor, for example, the division of the hour

from the part formatted as a list:

`{x = 10, y = 45; "One", "two", "three"}`

Exercises

Exercise 3.1. What will the following program print?

```
for i = -10, 10 do
  print(i, i% 3)
end
```

Exercise 3.2. What is the result of the expression $2 \wedge 3 \wedge 4$?

What about $2 \wedge -3 \wedge 4$?

Exercise 3.3. We can represent the polynomial

$$a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x^1 + a_0$$

in Lua as a list of its coefficients $\{a_0, a_1, \dots, a_n\}$.

Write a function that receives a polynomial (represent table) and x returns a polynomial value

ma in x .

Exercise 3.4. Can you write a function from the previous present exercises so as to use n additions and n multiplied scaling (and not using exponentiation)?

Exercise 3.5. How can you check if the value is boolean without resorting to the type function ?

Exercise 3.6. Consider the following expression:

`(x and y and (not z)) or ((not y) and x)`

Are parentheses needed in this expression? How would you go advised to use them in this expression?

Exercise 3.7. What will the following code snippet print?

Explain.

```
sunday = "monday"; monday = "sunday"  
t = {sunday = "monday", [sunday] = monday}  
print (t.sunday, t [sunday], t [t.sunday])
```

Exercise 3.8. Suppose you want to create a table that-paradise with each escape sequence (escape sequence) for strings binds its value. How would you write a constructor for such a table?

CHAPTER 4

Operators

Lua supports an almost traditional set of operators like to a set used in C or Pascal. Traditional operators include assignment, control constructs and calls procedures. Lua also supports the less common operators such as multiple assignment and locale definition variables.

4.1. Assignment operators

Assignment is the basic means of changing the values of variables noah and table fields:

```
a = "hello" .. "world"  
tn = tn + 1
```

Lua allows *multiple assignments* , which swarm assigns a list of values to a list of variables in one step.

For example, in the operator

```
a, b = 10, 2 * x
```

variable a gets the value 10 , and variable b gets the value $2 * x$.

In multiple assignment, Lua first evaluates all values.

and only then performs the assignments. Therefore, we can use use multiple assignments to change

in places two values, as in the following examples:

```
x, y = y, x  
- swap 'x' and 'y'  
a [i], a [j] = a [j], a [i] - swap 'a [i]' and 'a [j]'
```

Lua always converts the number of values to the number of variables: when the number of values is less than the number of variables, then